

Speaker Notes — Arquitectura de Software de Spotify

Slide 1: Portada (Westra)

Presentación del trabajo de investigación. Mostrar título, autores, universidad y curso.

Slide 2: Agenda (Westra)

Buenas. Hoy presentamos nuestra investigación sobre la arquitectura de software de Spotify y el rol del arquitecto de software. El trabajo se estructura en tres secciones: la arquitectura actual de Spotify, el estado del rol del arquitecto en el mercado laboral, y cómo evolucionará este rol en la próxima década.

Slide 3: Objetivo del Trabajo (Westra)

El objetivo de este trabajo es triple. Primero, investigar a fondo la arquitectura de Spotify, una de las plataformas de streaming más grandes del mundo. Segundo, analizar el rol actual del arquitecto de software mediante perfiles reales del mercado laboral. Y tercero, proyectar cómo evolucionará este rol en la próxima década frente a la inteligencia artificial y la computación cuántica.

Slide 4: Introducción (Westra)

Spotify es una de las plataformas de streaming de audio más grandes del mundo. Con más de 600 millones de usuarios activos mensuales y un catálogo superior a los 100 millones de pistas, representa un caso de estudio paradigmático sobre cómo una arquitectura bien diseñada habilita innovación continua y escalabilidad global.

Slide 5: Spotify en Cifras (Westra)

Estas cifras nos dan una idea de la escala a la que opera Spotify. Mil quinientos microservicios gestionados por más de 250 equipos autónomos, procesando más de un billón de eventos al día. Toda esta infraestructura corre sobre Google Cloud Platform desde 2018.

Términos clave:

- **Microservicio:** Servicio pequeño, autónomo e independiente que implementa una funcionalidad de negocio específica. Se despliega y escala de forma aislada.

Slide 6: Tipo de Arquitectura (Westra)

Entremos en la primera sección. Spotify opera bajo una arquitectura de microservicios con un modelo event-driven. La comunicación entre sus 1.500 servicios se resuelve mediante canales síncronos como REST y gRPC, y asíncronos con Apache Kafka como backbone principal. Utilizan Apollo GraphQL como API Gateway unificado para todos los clientes y aplican el patrón Strangler Fig para migrar progresivamente desde el monolito original.

Términos clave:

- **Arquitectura event-driven:** Estilo donde los componentes se comunican publicando y consumiendo eventos en lugar de llamadas directas. Favorece el desacoplamiento.
- **REST:** Estilo de API sobre HTTP que usa verbos (GET, POST, PUT, DELETE) para operar sobre recursos.
- **gRPC:** Framework de comunicación de alta eficiencia desarrollado por Google. Usa Protocol Buffers para serialización binaria, más rápido que REST con JSON.
- **API Gateway:** Punto único de entrada para todos los clientes que enruta peticiones a los microservicios internos y maneja autenticación, rate limiting y composición de respuestas.
- **Strangler Fig (patrón):** Estrategia de migración donde nuevas funcionalidades se construyen como microservicios y el tráfico se redirige incrementalmente desde el monolito hasta que este queda vacío.

Slide 7: Patrones Arquitectónicos (Daniel)

Entre los patrones identificados destacan CQRS para separar modelos de lectura y escritura en servicios de alta demanda como el catálogo y las playlists. Las transacciones distribuidas se coordinan mediante el patrón Saga con coreografía de eventos, sin un orquestador central. La distribución de contenido se apoya en una CDN multi-proveedor con puntos de presencia propios.

Términos clave:

- **CQRS (Command Query Responsibility Segregation):** Patrón que separa el modelo de escritura (comandos) del de lectura (consultas), permitiendo optimizar cada uno de forma independiente.
- **Saga / Coreografía:** Mecanismo para manejar transacciones distribuidas sin un orquestador central. Cada servicio publica eventos y los demás reaccionan; si algo falla, se emiten eventos compensatorios.
- **CDN (Content Delivery Network):** Red de servidores distribuidos globalmente que almacenan copias del contenido cerca de los usuarios para reducir latencia.
- **Edge Delivery:** Estrategia de servir contenido desde el punto de presencia más cercano al usuario, en el borde de la red.

Slide 8: Tecnologías Principales — Lenguajes (Daniel)

Spotify es notablemente políglota. Java con Spring Boot domina el backend de los microservicios core. Python, el lenguaje original del monolito, hoy se usa en machine learning. C++ maneja los componentes de baja latencia como el motor de streaming. Scala, mediante Scio, procesa los pipelines de datos a escala.

Términos clave:

- **Políglota (en arquitectura):** Enfoque donde cada componente usa el lenguaje o tecnología más adecuado para su caso de uso, en lugar de imponer uno único para todo el sistema.
- **Spring Boot:** Framework de Java que simplifica la creación de microservicios con configuración automática y servidor embebido.
- **Scio:** Librería desarrollada por Spotify sobre Apache Beam para procesamiento de datos a gran escala en Scala. Provee una API idiomática para pipelines batch y streaming.
- **Apache Beam:** Modelo unificado de programación para definir pipelines de procesamiento de datos que pueden ejecutarse en múltiples motores como Dataflow, Spark o Flink.

Slide 9: Tecnologías — Infraestructura (Daniel)

Entre 2016 y 2018 Spotify migró desde aproximadamente 5.000 servidores físicos en datacenters propios hacia Google Cloud Platform, en una de las migraciones a la nube más significativas de la industria. Hoy opera sobre Kubernetes con Docker, utiliza una CDN multi-proveedor y pipelines de CI/CD con Jenkins y Spinnaker. Para orquestación de batch jobs y flujos de datos utilizan Luigi y Styx, herramientas internas que automatizan procesos de ETL a escala global.

Términos clave:

- **Kubernetes (K8s):** Plataforma de código abierto para orquestar contenedores: automatiza despliegue, escalado y manejo de cargas de trabajo.
- **Docker:** Tecnología que empaqueta aplicaciones con todas sus dependencias en contenedores ligeros y portables que se ejecutan de forma aislada.
- **CI/CD (Continuous Integration / Continuous Delivery):** Prácticas de automatización: la integración continua valida cada cambio con pruebas automáticas; la entrega continua despliega a producción de forma automatizada.
- **Jenkins:** Servidor de automatización de código abierto para construir, probar y desplegar software (pipelines CI/CD).
- **Spinnaker:** Plataforma de entrega continua multi-cloud creada por Netflix y liberada como open source. Soporta despliegues canary, blue/green y rollbacks.
- **ETL (Extract, Transform, Load):** Proceso de mover datos desde fuentes externas, transformarlos a un formato adecuado y cargarlos en el destino. Crítico en pipelines de datos.

- **Luigi:** Framework de Python creado por Spotify para construir pipelines de datos complejos con manejo de dependencias, reintentos y visualización.
- **Styx:** Orquestador de workflows de datos desarrollado internamente por Spotify para ejecutar pipelines de ETL a escala.

Slide 10: Tecnologías — Almacenamiento y Mensajería (Daniel)

La estrategia de almacenamiento es poliglota: Cassandra con unos 3.000 nodos aloja los metadatos del catálogo, PostgreSQL gestiona datos relacionales, Redis y Memcached proporcionan la capa de caché esencial para latencias inferiores a 200 ms, y Elasticsearch indexa el catálogo para búsqueda. Apache Kafka es el sistema nervioso de la plataforma, procesando más de un billón de eventos al día.

Términos clave:

- **Apache Cassandra:** Base de datos NoSQL distribuida y masterless (sin nodo maestro) diseñada para alta disponibilidad, escalabilidad horizontal y replicación multirregión.
- **PostgreSQL:** Sistema de base de datos relacional de código abierto con soporte ACID, extensibilidad y fuerte consistencia. Ideal para datos estructurados como usuarios y facturación.
- **Redis:** Base de datos en memoria usada como caché distribuida y broker de mensajería. Proporciona latencias de microsegundos.
- **Memcached:** Sistema de caché distribuido en memoria para acelerar aplicaciones web reduciendo la carga sobre bases de datos.
- **Elasticsearch:** Motor de búsqueda y analítica distribuido basado en Apache Lucene. Indexa datos en tiempo real para búsquedas full-text.

Slide 11: Componentes de la Solución (Daniel)

La funcionalidad de Spotify se articula en ocho dominios de servicios core. Veamos brevemente cada uno.

Slide 12: Catálogo y Recomendaciones (Daniel)

El catálogo musical con más de 100 millones de pistas se apoya en Cassandra para metadatos y Elasticsearch para búsqueda. El motor de recomendaciones es el diferenciador competitivo más importante: BaRT equilibra explotación y exploración, y productos como Discover Weekly combinan filtrado colaborativo con procesamiento de lenguaje natural. Spotify utiliza un enfoque “Algotorial” que fusiona curaduría editorial humana con señales algorítmicas, logrando un balance entre relevancia computacional y criterio experto. En 2023 lanzaron DJ AI, que utiliza inteligencia artificial generativa y voz sintética.

Términos clave:

- **BaRT (Bandits for Recommendations as Treatments)**: Sistema de recomendación de Spotify basado en algoritmos de bandits multi-armados que balancean explotación (recomendar lo conocido) y exploración (descubrir nuevo contenido).
- **Filtrado colaborativo**: Técnica de recomendación que predice preferencias de un usuario basándose en patrones de usuarios similares. ‘A usuarios como tú también les gustó...’
- **Algotorial**: Término acuñado por Spotify para describir su enfoque híbrido que combina curaduría editorial humana con señales algorítmicas de machine learning.
- **Taste Profile**: Representación vectorial densa de las preferencias musicales de un usuario, generada por redes neuronales profundas a partir de su historial de escucha.
- **DJ AI**: Funcionalidad de Spotify (2023) que usa IA generativa y voz sintética para actuar como un curador personalizado que selecciona y comenta canciones.

Slide 13: Streaming y Publicidad (Daniel)

El streaming utiliza códecs OGG Vorbis y AAC con bitrate variable que se adapta dinámicamente a las condiciones de red. Spotify Connect sincroniza el estado entre dispositivos. En el nivel gratuito, el Ad Insertion Service inserta anuncios dinámicamente con segmentación avanzada y subasta en tiempo real.

Términos clave:

- **OGG Vorbis**: Códec de audio comprimido de código abierto y sin patentes. Ofrece mejor calidad que MP3 a la misma tasa de bits.
- **AAC (Advanced Audio Coding)**: Códec de audio con pérdida diseñado como sucesor del MP3. Usado por YouTube, Apple y Spotify.
- **VBR (Variable Bitrate)**: Técnica de codificación que ajusta dinámicamente la tasa de bits según la complejidad del audio, optimizando calidad vs. ancho de banda.
- **Spotify Connect**: Protocolo que permite sincronizar y transferir la reproducción entre dispositivos (teléfono, parlante, TV, auto) de forma fluida.

Slide 14: Social, Auth y Plataforma de Datos (Daniel)

Las playlists colaborativas utilizan CRDTs para permitir edición simultánea sin servidor central de coordinación. La plataforma ABBA permite ejecutar experimentos A/B a escala de cientos de millones de usuarios simultáneamente, siendo una de las infraestructuras de experimentación más grandes del mundo.

Términos clave:

- **CRDT (Conflict-free Replicated Data Type)**: Estructura de datos replicada que permite ediciones concurrentes por múltiples usuarios sin

necesidad de coordinación central. Usada en playlists colaborativas.

- **Experimentos A/B:** Metodología que compara dos versiones (A y B) de una funcionalidad con usuarios reales para medir cuál produce mejores resultados. Spotify ejecuta cientos simultáneamente mediante su plataforma ABBA.
- **Feature Store:** Repositorio centralizado de características (features) utilizado para entrenar y servir modelos de machine learning. Evita duplicación de lógica de transformación de datos.

Slide 15: Content Platform (Daniel)

Spotify opera también una plataforma de contenido completa. El Ingestion Pipeline automatiza la recepción de material desde sellos discográficos y distribuidores digitales. Spotify for Artists, Labels y Podcasters son los portales donde los creadores gestionan su presencia en la plataforma. El Rights Management gestiona los derechos de autor, licencias, y el cálculo y liquidación de regalías a los titulares de derechos — una pieza crítica del ecosistema.

Términos clave:

- **Ingestion Pipeline:** Flujo automatizado que recibe, valida, transforma y almacena contenido —en este caso, pistas de audio y metadatos desde sellos discográficos y distribuidores.

Slide 16: Evolución Histórica (Daniel)

La arquitectura de Spotify ha transitado por cinco fases diferenciadas. Partió de un monolito Python/Django en 2006, evolucionó hacia SOA con Cassandra y Kafka, migró integralmente a Google Cloud entre 2016 y 2018, maduró con GraphQL y Backstage, y hoy se expande con IA generativa y nuevos formatos como podcast y audiolibros.

Términos clave:

- **Monolito:** Aplicación donde toda la funcionalidad reside en un solo codebase y se despliega como una unidad. Simple al inicio, pero difícil de escalar con equipos grandes.
- **SOA (Service-Oriented Architecture):** Estilo arquitectónico donde las funcionalidades se dividen en servicios que se comunican mediante protocolos estándar. Precursor de los microservicios.

Slide 17: Manejo de Datos (Erick)

Spotify utiliza una arquitectura de datos distribuida donde cada motor se especializa en un tipo específico de información. Cada interacción del usuario genera eventos en Kafka que alimentan sistemas analíticos, motores de recomendación y procesos de machine learning, todo en tiempo real y sin afectar la experiencia del usuario.

Términos clave:

- **Machine Learning (ML):** Rama de la IA donde los sistemas aprenden patrones a partir de datos sin ser explícitamente programados. Spotify lo usa para recomendaciones, segmentación y personalización.

Slide 18: Seguridad (Erick)

La seguridad es un pilar fundamental. Spotify implementa OAuth 2.0 para autenticación, políticas de mínimo privilegio entre microservicios, cifrado AES-256 en reposo y TLS en tránsito, además de DRM para proteger los derechos de autor del contenido.

Términos clave:

- **OAuth 2.0:** Protocolo de autorización estándar (RFC 6749) que permite a aplicaciones acceder a recursos protegidos mediante tokens temporales sin exponer credenciales del usuario.
- **AES-256 (Advanced Encryption Standard):** Algoritmo de cifrado simétrico con clave de 256 bits. Estándar gubernamental para proteger datos en reposo. Virtualmente irrompible por fuerza bruta.
- **TLS (Transport Layer Security):** Protocolo criptográfico que garantiza comunicación segura sobre redes. Visible como HTTPS en navegadores. Cifra datos en tránsito y autentica servidores.
- **DRM (Digital Rights Management):** Tecnologías que controlan el acceso y uso de contenido digital protegido por derechos de autor. Limitan reproducción, copia y distribución no autorizada.
- **Mínimo privilegio (principio de):** Cada componente o usuario debe tener solo los permisos estrictamente necesarios para realizar su función. Reduce la superficie de ataque.

Slide 19: Escalabilidad y Disponibilidad (Erick)

Para la escalabilidad, Spotify utiliza autoescalado de Kubernetes, CDN global y sharding de bases de datos. Para la disponibilidad, replica datos en múltiples centros distribuidos geográficamente, Kubernetes reinicia servicios fallidos automáticamente, y herramientas como Prometheus y Grafana permiten monitoreo continuo y detección rápida de incidentes. En observabilidad, Spotify desarrolló herramientas internas como Heroic y Ffwd —liberadas como open source— para manejar series temporales a la escala de cientos de millones de usuarios.

Términos clave:

- **Autoescalado (autoscaling):** Capacidad de añadir o quitar automáticamente recursos computacionales según la demanda, optimizando costo y rendimiento. Kubernetes lo gestiona nativamente.
- **Sharding:** Técnica de particionado de bases de datos donde los datos se dividen en fragmentos (shards) distribuidos entre múltiples servidores.

Cada shard contiene un subconjunto de los datos.

- **Prometheus:** Sistema de monitoreo y alertas de código abierto que recolecta métricas en series temporales. Ampliamente usado en entornos cloud nativos.
- **Grafana:** Plataforma de visualización para métricas, logs y trazas. Se integra con Prometheus y permite crear dashboards interactivos y configurar alertas.
- **Heroic:** Sistema de series temporales desarrollado por Spotify sobre Cassandra y Elasticsearch. Liberado como open source en 2019.
- **Ffwd:** Herramienta interna de Spotify para manejo de series temporales a gran escala. También liberada como open source.

Slide 20: Diagrama de Arquitectura (Erick)

Este es el diagrama de arquitectura de Spotify, que muestra la estructura de microservicios, los flujos de datos y las integraciones entre los distintos componentes de la plataforma.

Slide 21: Diagrama de Arquitectura (2) (Erick)

La segunda parte del diagrama detalla componentes adicionales de la plataforma, incluyendo los sistemas de recomendación, procesamiento de datos y distribución de contenido.

Slide 22: ADR-1: Monolito → Microservicios — Decisiones Arquitectónicas Clave (Erick)

Ahora analizamos las decisiones arquitectónicas más importantes que dieron forma a Spotify, documentadas como Architecture Decision Records. Hacia 2012, un solo codebase impedía que múltiples equipos desplegaran de forma independiente. La descomposición en microservicios permitió que más de 250 squads desarrollaran en paralelo. La contrapartida fue una complejidad operativa significativa que Spotify gestionó construyendo herramientas como Backstage.

Términos clave:

- **ADR (Architecture Decision Record):** Documento breve que registra una decisión arquitectónica, su contexto, las alternativas consideradas y las consecuencias. Creado por Michael Nygard. Vive en el repositorio junto al código.
- **Codebase:** Conjunto completo de código fuente que compone un proyecto o sistema. En un monolito, todos los equipos comparten un solo codebase.
- **Backstage:** Portal de desarrolladores open source creado por Spotify (donado a la CNCF en 2020). Centraliza el catálogo de servicios, documentación viva y plantillas de scaffolding.

Slide 23: ADR-2: Google Cloud | ADR-3: Cassandra — Decisiones Arquitectónicas Clave (Erick)

Mantener datacenters propios desviaba talento hacia tareas de bajo valor. La migración a GCP permitió reducir costos y acceder a BigQuery y Dataflow. PostgreSQL no escalaba al ritmo del catálogo, así que Cassandra aportó escalabilidad horizontal sin punto único de fallo, aunque a costa de un modelo de consistencia eventual.

Términos clave:

- **TCO (Total Cost of Ownership):** Costo total de poseer y operar un sistema durante toda su vida útil: hardware, licencias, personal, energía, mantenimiento. Las decisiones cloud se evalúan con TCO, no solo con precio de servicio.
- **BigQuery:** Data warehouse serverless de Google Cloud que permite ejecutar consultas SQL analíticas sobre petabytes de datos en segundos. Spotify lo usa para analítica de negocio.
- **Dataflow:** Servicio gestionado de Google Cloud para ejecutar pipelines de Apache Beam. Procesa datos en tiempo real o batch sin administrar infraestructura.
- **Vendor lock-in:** Dependencia excesiva de un solo proveedor que dificulta migrar a otra plataforma. Riesgo inherente a las migraciones cloud que debe gestionarse.
- **Consistencia eventual:** Modelo donde las réplicas de datos se sincronizan con cierto retraso pero se garantiza que eventualmente convergerán al mismo estado. Opuesto a la consistencia fuerte (ACID).

Slide 24: ADR-4: Apache Kafka | ADR-5: Apollo GraphQL — Decisiones Arquitectónicas Clave (Erick)

Con cientos de microservicios, la comunicación punto a punto era insostenible. Kafka desacopla productores y consumidores y permite replay histórico de eventos. GraphQL permite que cada tipo de cliente declare exactamente los datos que necesita, optimizando ancho de banda y evitando over-fetching y under-fetching.

Términos clave:

- **Desacoplar:** Reducir dependencias entre componentes para que puedan evolucionar, fallar y escalar de forma independiente. Kafka desacopla productores de consumidores.
- **Over-fetching:** Problema de APIs REST donde el servidor devuelve más datos de los que el cliente necesita. GraphQL lo resuelve permitiendo al cliente declarar exactamente qué campos requiere.
- **Under-fetching:** Problema opuesto: el cliente recibe datos insuficientes y debe hacer múltiples llamadas adicionales (el ‘problema N+1’). GraphQL resuelve ambos en una sola consulta.

- **DataLoader:** Utilidad que agrupa y procesa por lotes las consultas a bases de datos para evitar el problema N+1 en resolvers de GraphQL.

Slide 25: ADR-6: Modelo de Squads | ADR-7: Backstage — Decisiones Arquitectónicas Clave (Erick)

Spotify adoptó el modelo de squads, tribes, chapters y guilds. Cada squad es dueño de una misión de negocio y de los microservicios que la implementan, bajo el principio “You build it, you run it”. Backstage unifica la visibilidad de todo el ecosistema en un solo lugar. Spotify lo donó a la CNCF en 2020, donde hoy es un proyecto incubado.

Términos clave:

- **Squad:** Equipo autónomo de 5-8 ingenieros dueño de una misión de negocio y de los microservicios que la implementan. Principio: ‘You build it, you run it’ (tú lo construyes, tú lo operas).
- **Tribe:** Agrupación de squads que trabajan en áreas relacionadas del producto. Límite informal de ~100 personas (principio de Dunbar).
- **Chapter:** Agrupación horizontal de personas con habilidades similares dentro de una tribe. El chapter lead combina rol técnico con mentoría.
- **Guild:** Comunidad de interés voluntaria que cruza toda la organización. Cualquiera puede unirse para compartir conocimiento sobre un tema (ej. testing, web, ML).
- **CNCF (Cloud Native Computing Foundation):** Fundación bajo la Linux Foundation que alberga proyectos cloud nativos como Kubernetes, Prometheus y Backstage. Gobierna el ecosistema open source cloud.
- **Scaffolding:** Generación automática de la estructura inicial de un proyecto (carpetas, configuraciones, dependencias). Backstage lo ofrece como plantillas para nuevos servicios.

Slide 26: ADR-8: Caché Distribuida (Redis + Memcached) — Decisiones Arquitectónicas Clave (Erick)

El último ADR documentado aborda la caché distribuida. Con latencias requeridas inferiores a 200 milisegundos para una experiencia de streaming fluida, consultar la base de datos en cada reproducción sería inviable. Redis y Memcached proporcionan una capa de caché que reduce la carga sobre Cassandra y PostgreSQL. La contrapartida es la inconsistencia temporal entre caché y base de datos, gestionada mediante tiempos de expiración y estrategias de invalidación diferenciadas por tipo de dato.

Términos clave:

- **TTL (Time To Live):** Tiempo de vida de un dato en caché antes de ser considerado obsoleto y eliminado o refrescado. Se usa para gestionar la inconsistencia temporal caché-BD.

- **Invalidación de caché:** Estrategia para marcar datos en caché como obsoletos cuando cambian en la base de datos. Es uno de los problemas más difíciles en sistemas distribuidos.

Slide 27: Conclusión (Erick)

En conclusión, el éxito de Spotify se sustenta en una arquitectura de microservicios basada en eventos, con tecnologías como Kubernetes, Kafka y Cassandra, y decisiones arquitectónicas bien documentadas. La alineación entre squads autónomos y microservicios es una manifestación concreta de la Ley de Conway.

Términos clave:

- **Ley de Conway:** Principio que establece que las organizaciones diseñan sistemas que reflejan su estructura de comunicación. En Spotify: squads autónomos → microservicios independientes.

Slide 28: Tabla Comparativa de Perfiles (1/3) (Michael)

Pasamos a la segunda sección, donde investigamos el estado actual del rol del arquitecto de software mediante perfiles reales del mercado laboral.

Spotify — Staff Software Architect (Suecia, Híbrido): Define la visión técnica a largo plazo de los sistemas de distribución de contenido, guía equipos multidisciplinares en decisiones arquitectónicas y mitiga riesgos técnicos en plataformas de alta disponibilidad.

Globant — Software Architect Cloud Solutions (EE.UU., Remoto): Diseña y migra arquitecturas empresariales hacia entornos multi-nube, asegura gobernanza de datos y costos de infraestructura (FinOps), y evalúa y audita la seguridad de soluciones integradas.

Mercado Libre — Arquitecto de Software Senior (Argentina, Híbrido): Diseña arquitecturas orientadas a microservicios tolerantes a fallos, optimiza bases de datos distribuidas y flujos de mensajería masiva, y mentoriza ingenieros senior definiendo estándares de calidad de código.

Fuentes: LinkedIn, Glassdoor, Get on Board (2026).

Slide 29: Tabla Comparativa de Perfiles (2/3) (Michael)

Habilidades Técnicas:

- Spotify: Diseño de sistemas distribuidos a gran escala (hiperescala), modelado de microservicios y arquitecturas de eventos, gestión de resiliencia y tolerancia a fallos.
- Globant: Arquitectura cloud nativa (IaaS, PaaS, SaaS), automatización de infraestructura como código (IaC), estrategias de seguridad perimetral y gobierno de datos.

- Mercado Libre: Patrones de diseño de software, optimización de consultas distribuidas, estrategias de caché e ingesta de datos en tiempo real, pruebas de carga y análisis de rendimiento.

Habilidades Blandas:

- Spotify: Liderazgo de influencia sin autoridad formal, comunicación estratégica y negociación con directivos, gestión y resolución de conflictos técnicos.
- Globant: Comunicación técnica clara a audiencias no técnicas, adaptabilidad a entornos cambiantes y prioridades de clientes, pensamiento analítico orientado a objetivos de negocio.
- Mercado Libre: Mentoría, empatía y desarrollo de talento técnico, trabajo en equipo colaborativo bajo metodologías ágiles, proactividad y toma de decisiones bajo presión.

Términos clave:

- **IaC (Infrastructure as Code):** Práctica de gestionar y aprovisionar infraestructura mediante archivos de configuración versionables.

Slide 30: Tabla Comparativa de Perfiles (3/3) (Michael)

Tecnologías mencionadas:

- Spotify: Java, C++, Python, GCP, Kubernetes, Docker, Kafka, gRPC. Stack orientado a hiperescala sobre Google Cloud.
- Globant: AWS, Azure, Terraform, Jenkins, Docker, SQL Server, PostgreSQL, Redis. Stack multi-cloud con foco en automatización.
- Mercado Libre: Go, Java, Python, AWS, Elasticsearch, Redis, MySQL, Apache Kafka. Stack regional sobre AWS con énfasis en datos y mensajería.

Brecha salarial: Norteamérica/Europa (\$120K-\$185K) vs Latinoamérica (\$60K-\$75K). Las exigencias técnicas conceptuales son muy similares, pero las compensaciones están ligadas a la ubicación geográfica y el costo de vida local de la empresa contratante.

Slide 31: Coincidencias Clave (Michael)

Las tres coincidencias principales son: dominio obligatorio de cloud y contenedores en todas las geografías; experiencia en Kafka y bases distribuidas como requisito transversal; y la importancia extrema de las habilidades blandas. A diferencia de roles puramente técnicos, el arquitecto pasa gran parte de su tiempo comunicando, negociando e influenciando.

Slide 32: Diferencias Significativas (Michael)

Las principales diferencias están en el contexto: Spotify se enfoca en hiperescala y resiliencia, mientras Globant tiene un matiz de consultoría y gobernanza de costos.

La brecha salarial es marcada: entre 120 y 185 mil dólares en Norteamérica y Europa frente a 60-75 mil en Latinoamérica, pese a que las exigencias técnicas son muy similares.

Términos clave:

- **FinOps (Financial Operations):** Práctica que une finanzas, tecnología y negocio para gestionar y optimizar costos en la nube. El arquitecto debe entender el impacto económico de sus decisiones técnicas.

Slide 33: Conclusión (Michael)

El arquitecto de software ha evolucionado más allá del diseño técnico hacia una posición estratégica. Las empresas buscan profesionales con conocimientos en microservicios, cloud, DevOps y ciberseguridad, pero las habilidades blandas son igualmente valoradas. El arquitecto funciona como puente entre los objetivos del negocio y la implementación tecnológica.

Términos clave:

- **DevOps:** Cultura y práctica que integra desarrollo (Dev) y operaciones (Ops) para acortar ciclos de entrega. El arquitecto debe diseñar sistemas que faciliten CI/CD, monitoreo y operación autónoma.

Slide 34: Transformación del Rol (Michael)

En la tercera sección proyectamos la evolución del rol del arquitecto en la próxima década. Durante los próximos diez años, el arquitecto evolucionará de ser principalmente un diseñador técnico a convertirse en un estratega tecnológico. Deberá comprender no solo la infraestructura, sino también el impacto ético, económico, ambiental y social de sus decisiones.

Slide 35: Impacto de la IA (Charlie)

La IA transformará directamente las actividades del arquitecto. Podrá analizar grandes cantidades de información, detectar deuda técnica, generar documentación y sugerir mejoras. Sin embargo, la IA generativa todavía enfrenta problemas de precisión, alucinaciones y falta de marcos sólidos de evaluación. La IA debe verse como herramienta de apoyo, no como reemplazo del arquitecto. Fuente: Esposito et al., 2025; Software Engineering Institute, 2024.

Términos clave:

- **Alucinación (en IA):** Fenómeno donde un modelo de IA generativa produce información incorrecta o inventada presentada como factual. Riesgo crítico al usar IA para decisiones arquitectónicas.

Slide 36: Impacto de la Computación Cuántica (Charlie)

La computación cuántica podría influir en optimización y simulación en la próxima década. IBM proyecta una computadora cuántica tolerante a fallos hacia 2029. El impacto más inmediato estará en seguridad: el arquitecto deberá planificar migraciones hacia algoritmos resistentes a ataques cuánticos. El NIST ya publicó estándares de criptografía post-cuántica en 2024.

Términos clave:

- **Criptografía post-cuántica:** Algoritmos criptográficos diseñados para resistir ataques de computadoras cuánticas. Estandarizados por el NIST en 2024. Reemplazarán a RSA y ECC en el futuro.
- **NIST (National Institute of Standards and Technology):** Agencia del gobierno de EE.UU. que desarrolla estándares tecnológicos. Publicó los primeros estándares de criptografía post-cuántica en 2024.

Slide 37: Nuevas Responsabilidades (Charlie)

El arquitecto del futuro tendrá responsabilidades más amplias: gobernanza de sistemas de IA bajo ISO 42001, sostenibilidad medida con la especificación SCI de la Green Software Foundation, ética tecnológica y cumplimiento normativo. Ya no bastará con que el sistema funcione; deberá ser auditable, confiable y alineado con principios de privacidad y transparencia.

Términos clave:

- **ISO/IEC 42001:2023:** Estándar internacional para sistemas de gestión de inteligencia artificial. Establece requisitos para gobernar, monitorear y mejorar continuamente sistemas de IA.
- **SCI (Software Carbon Intensity):** Especificación de la Green Software Foundation que mide la tasa de emisiones de carbono de un sistema de software. Permite tomar decisiones de diseño más sostenibles.
- **Green Software Foundation:** Organización sin fines de lucro que desarrolla estándares y herramientas para reducir el impacto ambiental del software.

Slide 38: Habilidades a Desarrollar (Charlie)

El arquitecto necesitará una combinación de habilidades técnicas y humanas. En lo técnico: cloud, MLOps, gobernanza de datos, IA generativa y fundamentos de criptografía post-cuántica. En lo humano: pensamiento crítico, liderazgo, negociación y capacidad para analizar trade-offs entre rendimiento, costo, privacidad y sostenibilidad.

Términos clave:

- **MLOps (Machine Learning Operations):** Conjunto de prácticas que aplican principios DevOps al ciclo de vida de modelos ML: integración

continua, entrega continua y monitoreo continuo de modelos en producción.

- **Trade-off (compensación):** Sacrificio aceptado en un aspecto para ganar en otro. En arquitectura: rendimiento vs. costo, personalización vs. privacidad, velocidad vs. estabilidad. No hay decisiones perfectas.

Slide 39: Automatización vs. Criterio Humano (Charlie)

Varias actividades podrán automatizarse: documentación, diagramas, análisis de dependencias, detección de deuda técnica. Pero el criterio humano seguirá siendo indispensable para prioridades estratégicas, resolver conflictos, evaluar consecuencias éticas y decidir sobre seguridad y migraciones complejas. El arquitecto actuará como mediador entre la automatización y la responsabilidad.

Términos clave:

- **Deuda técnica:** Costo implícito de retrabajo causado por elegir soluciones rápidas o subóptimas en lugar de enfoques de mayor calidad. La IA puede acelerar su acumulación si no se revisa.

Slide 40: Valor Diferencial del Arquitecto (Charlie)

El valor diferencial del arquitecto estará en conectar tecnología, negocio y responsabilidad social. Mientras la IA podrá generar opciones, el arquitecto deberá formular las preguntas correctas, evaluar riesgos y asegurar que la arquitectura sea sostenible en el tiempo. No será solo un experto técnico, sino un líder de decisiones complejas.

Slide 41: Riesgos y Oportunidades (Charlie)

Las oportunidades incluyen herramientas más avanzadas, automatización de tareas repetitivas y más tiempo para lo estratégico. Los riesgos abarcan la dependencia excesiva de la IA sin validación, generación de deuda técnica a gran velocidad, sesgos algorítmicos y aumento del consumo energético.

Términos clave:

- **Sesgo algorítmico:** Error sistemático en un modelo de IA que produce resultados injustos o discriminatorios hacia ciertos grupos. El arquitecto debe diseñar sistemas que mitiguen sesgos en datos y modelos.

Slide 42: Conclusión (Charlie)

En conclusión, el rol del arquitecto de software no desaparecerá. Se volverá más estratégico, interdisciplinario y crítico. La diferencia no estará solo en conocer nuevas tecnologías, sino en saber cuándo usarlas, cómo gobernarlas y qué consecuencias pueden generar.

Slide 43: Síntesis del Trabajo (Westra)

Llegamos a la conclusión general de nuestra investigación. Nuestro trabajo evidencia que el éxito de Spotify reside en una arquitectura cuidadosamente diseñada que equilibra escalabilidad, disponibilidad, seguridad y capacidad de innovación. El arquitecto moderno trasciende lo técnico para convertirse en puente estratégico. Y el futuro exigirá nuevas competencias sin eliminar la necesidad del criterio humano.

Slide 44: Referencias (Westra)

Estas son las principales referencias consultadas para este trabajo. Se utilizaron fuentes académicas, técnicas y de la industria, incluyendo papers originales de ingenieros de Spotify, documentación oficial de tecnologías como Kafka y Cassandra, y estándares recientes de ISO y NIST. La lista completa de referencias en formato APA se encuentra en el documento de investigación.

Términos clave:

- **APA (American Psychological Association):** Estilo de citación y formato académico estándar. Incluye citas autor-año en el texto y referencias completas al final. Requerido por el SOW del trabajo.

Slide 45: ¡Gracias! (Westra)

Muchas gracias por su atención. Quedamos abiertos a preguntas y comentarios.